



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Diseño e implementación de
un sistema SIEM ligero de un
Polvorín Militar simulado
Documentación Técnica**



Presentado por Yobana Nido Álvarez
en Universidad de Burgos — 8 de junio
de 2026

Tutor: José Manuel Sáiz Díez

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	5
Apéndice B Especificación de Requisitos	9
B.1. Introducción	9
B.2. Objetivos generales	9
B.3. Catálogo de requisitos	10
B.4. Especificación de requisitos	11
Apéndice C Especificación de diseño	15
C.1. Introducción	15
C.2. Diseño de datos	15
C.3. Diseño arquitectónico	16
C.4. Diseño procedimental	19
Apéndice D Documentación técnica de programación	21
D.1. Introducción	21
D.2. Estructura de directorios	22
D.3. Manual del programador	24

D.4. Compilación, instalación y ejecución del proyecto	32
D.5. Pruebas del sistema	34
Apéndice E Documentación de usuario	41
E.1. Introducción	41
E.2. Requisitos de usuarios	41
E.3. Instalación	42
E.4. Manual del usuario	42
Apéndice F Anexo de sostenibilización curricular	47
F.1. Introducción	47
Bibliografía	51

Índice de figuras

A.1. Diagrama de Gantt del desarrollo del proyecto.	3
B.1. Diagrama de casos de uso del sistema.	12
C.1. Contenido de la tabla <i>events</i> almacenada en SQLite.	16
C.2. Contenido de la tabla <i>alerts</i> almacenada en SQLite.	16
C.3. Arquitectura general del sistema SIEM desarrollado.	18
C.4. Ejecución del sistema SIEM durante el procesamiento de eventos.	20
D.1. Estructura general del proyecto visualizada desde Visual Studio Code.	23
D.2. Ejecución del módulo de ingesta durante las pruebas realizadas.	26
D.3. Implementación de las reglas principales del motor de correlación.	27
D.4. Contenido de la tabla <i>events</i> almacenada en SQLite.	28
D.5. Contenido de la tabla <i>alerts</i> almacenada en SQLite.	28
D.6. Resultado de la detección de anomalías clasificadas según su criticidad y nivel de riesgo.	29
D.7. Documentación interactiva de la API REST generada mediante Swagger.	30
D.8. Consulta realizada mediante el endpoint <i>/alerts</i>	31
D.9. Vista principal del dashboard de monitorización.	32
D.10. Visualización de alertas generadas por el sistema.	32
D.11. Ejecución del script de arranque <i>start.bat</i>	33
D.12. Ejecución del módulo de ingesta durante las pruebas realizadas.	35
D.13. Documentación Swagger generada automáticamente por FastAPI.	35
D.14. Consulta de eventos realizada mediante Swagger.	36
D.15. Contenido de la tabla <i>events</i> almacenada en SQLite.	36
D.16. Contenido de la tabla <i>alerts</i> almacenada en SQLite.	37

D.17.Resultado de la detección de anomalías clasificadas según su nivel de criticidad y nivel de riesgo.	37
D.18.Vista principal del dashboard.	38
D.19.Visualización de alertas generadas por el sistema.	38
D.20.Visualización de eventos almacenados.	38
D.21.Resultado del análisis estático realizado mediante Codacy.	39
E.1. Vista principal del dashboard del sistema SIEM.	43
E.2. Consulta de alertas generadas por el sistema.	43
E.3. Consulta de eventos almacenados en la base de datos.	44
E.4. Monitorización del estado de los sensores.	44
E.5. Visualización de anomalías detectadas por el sistema.	45
E.6. Panel de configuración del sistema.	45
E.7. Configuración de la simulación de notificaciones SMS.	46

Índice de tablas

A.1. Planificación temporal aproximada del desarrollo del proyecto. . .	2
A.2. Estimación temporal de las principales fases del proyecto.	3
A.3. Evolución temporal aproximada del desarrollo del proyecto. . . .	4
A.4. Estimación económica del proyecto.	6
B.1. Casos de uso principales del sistema.	11
B.2. Caso de uso CU-01.	12
B.3. Caso de uso CU-02.	13
B.4. Caso de uso CU-03.	13
B.5. Caso de uso CU-04.	13
B.6. Caso de uso CU-05.	14
B.7. Caso de uso CU-06.	14

Apéndice A

Plan de Proyecto Software

A.1. Introducción

El presente anexo recoge los aspectos relacionados con la planificación y viabilidad del proyecto desarrollado. Se describen las principales fases de trabajo realizadas durante el desarrollo del sistema SIEM, así como una estimación de los costes asociados y los aspectos legales considerados.

Aunque el proyecto se ha desarrollado con fines académicos, resulta interesante analizar su viabilidad desde una perspectiva similar a la utilizada en proyectos profesionales de desarrollo software.

A.2. Planificación temporal

El desarrollo del proyecto se llevó a cabo entre los meses de marzo y junio de 2026 siguiendo una planificación incremental. Durante este periodo se fueron implementando de forma progresiva los diferentes módulos que componen el sistema SIEM, realizando pruebas de integración y mejoras continuas sobre cada uno de ellos.

La Tabla [A.1](#) muestra la distribución temporal aproximada de las principales actividades realizadas durante el desarrollo del proyecto.

Semana	Actividades realizadas
1 (23-29 marzo)	Revisión de la arquitectura inicial del proyecto, organización del repositorio y preparación del entorno de trabajo.
2 (30 marzo-5 abril)	Definición del entorno simulado y revisión del formato de eventos utilizado por el sistema.
3 (6-12 abril)	Desarrollo y validación del módulo de ingesta de eventos.
4 (13-19 abril)	Implementación de la base de datos SQLite y almacenamiento persistente de eventos.
5 (20-26 abril)	Desarrollo del motor de correlación y generación de alertas.
6 (27 abril-3 mayo)	Incorporación de reglas de detección de anomalías y validación de resultados.
7 (4-10 mayo)	Desarrollo de la API REST mediante FastAPI.
8 (11-17 mayo)	Desarrollo inicial del dashboard de monitorización con Streamlit.
9 (18-24 mayo)	Implementación de estadísticas, visualizaciones gráficas y consulta de eventos y alertas.
10 (25-31 mayo)	Desarrollo de la monitorización de sensores, configuración del sistema y pruebas funcionales.
11 (1-7 junio)	Revisión general del sistema, mejora de la calidad del código y validación final.
12 (8-11 junio)	Redacción final de la memoria, elaboración de anexos y preparación de la defensa del proyecto.

Tabla A.1: Planificación temporal aproximada del desarrollo del proyecto.

Las principales fases del proyecto y la dedicación estimada asociada a cada una de ellas se muestran en la Tabla A.2.

La figura muestra de forma gráfica la planificación seguida durante el desarrollo del proyecto, indicando la distribución temporal aproximada de las diferentes tareas realizadas.

Fase	Horas estimadas
Análisis y definición de requisitos	25
Diseño de la arquitectura	25
Desarrollo del módulo de ingesta	35
Implementación de la base de datos	20
Desarrollo del motor de correlación	45
Detección de anomalías	30
Desarrollo de la API REST	30
Desarrollo del dashboard	50
Pruebas y validación	40
Documentación final	50
Total	350

Tabla A.2: Estimación temporal de las principales fases del proyecto.

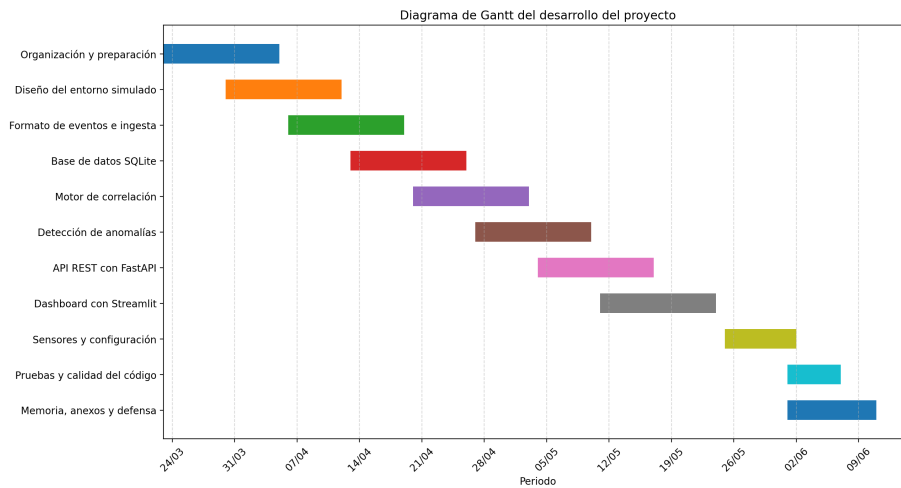


Figura A.1: Diagrama de Gantt del desarrollo del proyecto.

0.95

El desarrollo del proyecto se realizó de forma progresiva, comenzando por la definición de la arquitectura y los mecanismos de ingesta de eventos. Posteriormente se implementaron los módulos de almacenamiento, correlación y detección de anomalías, finalizando con el desarrollo del dashboard de monitorización, las tareas de validación y la elaboración de la documentación del proyecto.

Seguimiento del proyecto

Además de la planificación inicial, durante el desarrollo se realizó un seguimiento continuo de las tareas mediante el uso de GitHub, utilizando incidencias (issues) y hitos (milestones) para organizar las diferentes fases del proyecto.

La Tabla A.2 muestra una estimación de la evolución temporal seguida durante el desarrollo del sistema.

Fase	Periodo aproximado
Definición de objetivos y organización inicial	Marzo - Abril 2026
Diseño de la arquitectura del sistema	Abril 2026
Desarrollo del módulo de ingesta	Abril 2026
Implementación de la base de datos SQLite	Abril 2026
Desarrollo del motor de correlación	Abril - Mayo 2026
Implementación de la supervisión de sensores	Mayo 2026
Implementación de detección de anomalías	Mayo 2026
Desarrollo de la API REST	Mayo 2026
Desarrollo del dashboard web	Mayo 2026
Pruebas e integración de componentes	Mayo 2026
Documentación técnica y memoria final	Mayo - Junio 2026

Tabla A.3: Evolución temporal aproximada del desarrollo del proyecto.

Durante el desarrollo se establecieron diferentes hitos que permitieron organizar el trabajo de forma incremental. La primera fase estuvo centrada en la preparación del repositorio, la definición de la estructura general del proyecto y la implementación inicial del módulo de ingesta. Posteriormente se desarrollaron los componentes principales del sistema SIEM, incluyendo el almacenamiento en SQLite, el motor de correlación y la monitorización de sensores.

En una tercera fase se incorporaron funcionalidades avanzadas como la API REST desarrollada con FastAPI, el dashboard web implementado con Streamlit y los mecanismos de detección de anomalías. Finalmente, se realizaron las pruebas de integración y la elaboración de la documentación asociada al proyecto.

La utilización de GitHub permitió registrar el progreso del proyecto mediante incidencias y hitos, facilitando el seguimiento de las tareas realizadas y la validación progresiva de cada uno de los componentes desarrollados.

- Análisis inicial de requisitos y definición de objetivos.

- Diseño de la arquitectura general del sistema.
- Diseño del entorno simulado del polvorín.
- Definición del formato de eventos y logs.
- Desarrollo del módulo de ingesta.
- Implementación de la base de datos SQLite.
- Desarrollo del motor de correlación.
- Implementación del módulo de detección de anomalías.
- Desarrollo de la API REST mediante FastAPI.
- Desarrollo del dashboard de monitorización mediante Streamlit.
- Pruebas, validación y documentación final.

Durante el desarrollo fue necesario realizar diversas modificaciones sobre la planificación inicial, especialmente en las fases relacionadas con la correlación de eventos, la detección de anomalías y el desarrollo del dashboard. La naturaleza incremental del proyecto permitió adaptar el alcance de cada fase en función de los resultados obtenidos y de los requisitos surgidos durante el desarrollo.

La naturaleza incremental del proyecto permitió validar cada módulo de forma independiente antes de proceder a su integración con el resto de componentes.

A.3. Estudio de viabilidad

Antes de iniciar el desarrollo del proyecto se realizó un análisis preliminar para determinar si la solución propuesta era viable desde el punto de vista técnico, económico y legal.

El objetivo de este estudio fue comprobar que las tecnologías seleccionadas permitían desarrollar un sistema SIEM funcional dentro de las limitaciones temporales y de recursos propias de un Trabajo Fin de Grado.

Tras analizar las herramientas disponibles y los requisitos planteados, se concluyó que el proyecto era viable utilizando tecnologías de código abierto, una arquitectura ligera y un entorno simulado que permitiera validar el

funcionamiento del sistema sin necesidad de disponer de infraestructuras reales.

A continuación se analizan los aspectos económicos y legales más relevantes relacionados con el desarrollo del proyecto.

Viabilidad económica

Desde el punto de vista económico, el proyecto presenta una elevada viabilidad debido al uso de tecnologías de código abierto y herramientas gratuitas.

Las principales tecnologías utilizadas durante el desarrollo han sido Python, SQLite, FastAPI, Streamlit, Git y GitHub, todas ellas disponibles sin costes de licencia.

La estimación económica más relevante corresponde al tiempo de desarrollo invertido. Considerando un escenario profesional y una dedicación aproximada de 350 horas de trabajo, incluyendo análisis, desarrollo, pruebas, documentación y elaboración de la memoria, y tomando como referencia un coste de 20 euros por hora, el coste estimado del proyecto ascendería aproximadamente a 7.000 euros.

Concepto	Coste estimado (€)
Desarrollo software (350 h × 20 €/h)	7.000
Licencias software	0
Base de datos SQLite	0
FastAPI y Streamlit	0
Git y GitHub	0
Coste total estimado	7.000

Tabla A.4: Estimación económica del proyecto.

La distribución temporal presentada constituye una estimación aproximada del esfuerzo realizado en cada fase del proyecto. Debido a la naturaleza iterativa del desarrollo, algunas tareas fueron revisadas y ampliadas conforme avanzaba la implementación del sistema.

No obstante, al tratarse de un proyecto académico desarrollado con herramientas gratuitas y sin necesidad de infraestructuras específicas, los costes reales han sido mínimos.

Viabilidad legal

El proyecto ha sido desarrollado utilizando exclusivamente tecnologías de código abierto o herramientas con licencias gratuitas para uso académico.

El entorno utilizado corresponde a una simulación ficticia de un polvorín militar y no incorpora información sensible, procedimientos operativos reales ni documentación clasificada.

Asimismo, las imágenes y recursos gráficos utilizados en la memoria han sido generados por el autor o mediante herramientas de inteligencia artificial utilizadas como apoyo al desarrollo, indicando expresamente su utilización cuando ha sido necesario.

Por todo ello, no se identifican impedimentos legales para el desarrollo y utilización académica del sistema presentado en este Trabajo Fin de Grado.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este anexo se recogen los requisitos funcionales y no funcionales definidos para el desarrollo del sistema SIEM. Asimismo, se describen los principales casos de uso que permiten comprender las funcionalidades implementadas y la interacción del usuario con el sistema.

La especificación de requisitos ha servido como guía durante todo el proceso de desarrollo, permitiendo verificar posteriormente que los objetivos planteados inicialmente han sido alcanzados.

B.2. Objetivos generales

Los objetivos generales definidos para el sistema son los siguientes:

- Centralizar la información procedente de diferentes dispositivos de seguridad.
- Almacenar de forma persistente los eventos generados por el entorno monitorizado.
- Detectar situaciones potencialmente peligrosas mediante reglas de correlación.
- Identificar anomalías relacionadas con accesos, sensores y condiciones ambientales.

- Facilitar la consulta de información mediante una API REST.
- Proporcionar una interfaz gráfica que permita supervisar el estado del sistema.
- Mantener una arquitectura modular que facilite futuras ampliaciones.

B.3. Catálogo de requisitos

Requisitos funcionales

- **RF-01:** El sistema deberá procesar eventos procedentes de diferentes dispositivos simulados.
- **RF-02:** El sistema deberá almacenar los eventos en una base de datos.
- **RF-03:** El sistema deberá generar alertas mediante reglas de correlación.
- **RF-04:** El sistema deberá detectar anomalías sobre los eventos almacenados.
- **RF-05:** El sistema deberá proporcionar una API REST para la consulta de información.
- **RF-06:** El sistema deberá mostrar eventos y alertas mediante un dashboard web.
- **RF-07:** El sistema deberá mostrar el estado de los sensores monitorizados.
- **RF-08:** El sistema deberá permitir la visualización de estadísticas de funcionamiento.
- **RF-09:** El sistema deberá clasificar las anomalías según su nivel de riesgo.
- **RF-10:** El sistema deberá permitir la configuración de determinados parámetros operativos.

Requisitos no funcionales

- **RNF-01:** El sistema deberá ser modular y fácilmente mantenible.
- **RNF-02:** El sistema deberá utilizar tecnologías de código abierto.
- **RNF-03:** El sistema deberá ejecutarse con un consumo reducido de recursos.
- **RNF-04:** El sistema deberá ofrecer una interfaz sencilla e intuitiva.
- **RNF-05:** El sistema deberá facilitar futuras ampliaciones funcionales.

B.4. Especificación de requisitos

Código	Caso de uso
CU-01	Consultar dashboard principal
CU-02	Consultar eventos registrados
CU-03	Consultar alertas generadas
CU-04	Consultar anomalías detectadas
CU-05	Consultar estado de sensores
CU-06	Configurar parámetros del sistema

Tabla B.1: Casos de uso principales del sistema.

Los casos de uso anteriores representan las principales funcionalidades ofrecidas por el sistema al usuario. Todas ellas se encuentran accesibles a través del dashboard web desarrollado con Streamlit.

Diagrama de casos de uso

La Figura [B.1](#) muestra los principales casos de uso implementados en el sistema. Todas las funcionalidades son accesibles desde la interfaz web y permiten al usuario supervisar el estado general del SIEM, consultar la información registrada y configurar determinados parámetros operativos.

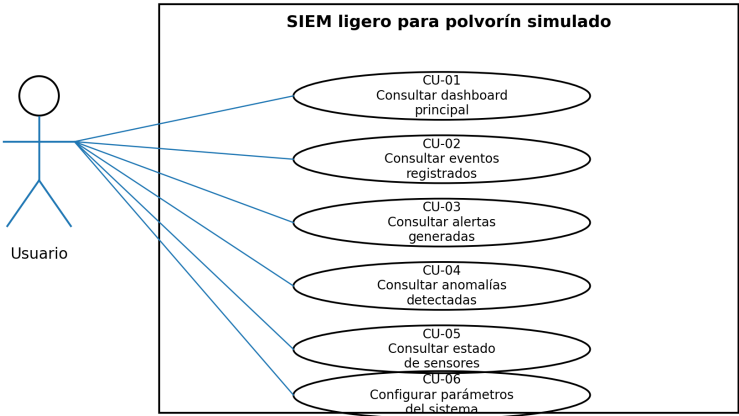


Figura B.1: Diagrama de casos de uso del sistema.

CU-01 Consultar dashboard principal

Código	CU-01
Nombre	Consultar dashboard principal
Requisitos asociados	RF-06, RF-08
Descripción	Permite visualizar el estado general del sistema, estadísticas y actividad reciente.
Actor	Usuario
Precondición	El dashboard se encuentra en ejecución.
Postcondición	El usuario obtiene una visión general del estado del sistema.

Tabla B.2: Caso de uso CU-01.

Código	CU-02
Nombre	Consultar eventos registrados
Requisitos asociados	RF-01, RF-02, RF-05
Descripción	Permite visualizar los eventos almacenados por el sistema y realizar consultas sobre ellos.
Actor	Usuario
Precondición	Existen eventos almacenados en la base de datos.
Postcondición	El usuario puede analizar la información registrada.

Tabla B.3: Caso de uso CU-02.

Código	CU-03
Nombre	Consultar alertas generadas
Requisitos asociados	RF-03, RF-04, RF-05
Descripción	Permite visualizar las alertas generadas por el motor de correlación, incluyendo su nivel de criticidad, fecha y origen.
Actor	Usuario
Precondición	Existen alertas registradas en la base de datos.
Postcondición	El usuario puede analizar las alertas y evaluar posibles incidentes de seguridad.

Tabla B.4: Caso de uso CU-03.

Código	CU-04
Nombre	Consultar anomalías detectadas
Requisitos asociados	RF-03, RF-07
Descripción	Permite visualizar comportamientos anómalos detectados por las reglas de correlación configuradas en el sistema.
Actor	Usuario
Precondición	El sistema ha detectado una o más anomalías.
Postcondición	El usuario puede revisar los eventos asociados y analizar la situación.

Tabla B.5: Caso de uso CU-04.

Código	CU-05
Nombre	Consultar estado de sensores
Requisitos asociados	RF-09
Descripción	Permite visualizar el estado operativo de los sensores monitorizados por el sistema, identificando dispositivos activos e inactivos.
Actor	Usuario
Precondición	El módulo de supervisión de sensores se encuentra operativo.
Postcondición	El usuario conoce el estado de disponibilidad de cada sensor.

Tabla B.6: Caso de uso CU-05.

CU-02 Consultar eventos registrados

CU-03 Consultar alertas generadas

CU-04 Consultar anomalías detectadas

CU-05 Consultar estado de sensores

CU-06 Configurar parámetros del sistema

Código	CU-06
Nombre	Configurar parámetros del sistema
Requisitos asociados	RF-10
Descripción	Permite modificar determinados parámetros operativos del sistema, como umbrales o configuraciones de monitorización.
Actor	Usuario
Precondición	El sistema se encuentra en funcionamiento.
Postcondición	Los nuevos parámetros quedan aplicados.

Tabla B.7: Caso de uso CU-06.

Apéndice C

Especificación de diseño

C.1. Introducción

En este anexo se describen los aspectos relacionados con el diseño del sistema desarrollado. Se detallan la estructura de los datos utilizados, la arquitectura general de la solución y los principales procesos implementados para el tratamiento de la información.

Asimismo, se presentan los diferentes componentes que integran el sistema SIEM, así como las relaciones existentes entre ellos y el flujo de procesamiento seguido desde la generación de los eventos hasta su visualización por parte del usuario.

El objetivo es proporcionar una visión técnica de cómo se ha diseñado el sistema y de las decisiones adoptadas durante su desarrollo.

C.2. Diseño de datos

El sistema utiliza una base de datos SQLite para almacenar de forma persistente la información generada durante el funcionamiento del SIEM.

Las entidades principales utilizadas son los eventos y las alertas. Los eventos representan las acciones o sucesos detectados por los distintos dispositivos monitorizados, mientras que las alertas corresponden a situaciones generadas por el motor de correlación cuando se cumplen determinadas condiciones de seguridad.

La tabla de eventos almacena información relacionada con la fecha y hora del suceso, origen del evento, tipo de evento, criticidad, usuario implicado,

punto de acceso, depósito afectado, dispositivo generador, resultado de la operación y descripción del evento.

Por su parte, la tabla de alertas almacena la información generada por las reglas de correlación y los mecanismos de detección implementados en el sistema.

El diseño adoptado permite almacenar la información de forma sencilla y eficiente, facilitando las consultas posteriores realizadas por la API y el dashboard.

La base de datos SQLite implementada en el proyecto se compone principalmente de las tablas *events* y *alerts*, encargadas de almacenar los eventos procesados por el sistema y las alertas generadas por el motor de correlación.

La Figura C.X y la Figura C.X muestran ejemplos del contenido almacenado en las tablas principales utilizadas por el sistema durante las pruebas realizadas.

id	timestamp	source	event_type	severity	user_id	access_point	deposit_id	device_id	result
1	2026-01-15 08:10:00	lector_tarjeta_d01	access	INFO	U001	deposito_01	D01	lector_d01	allo
2	2026-02-12 09:20:00	sensor_temp_d04	environment	WARNING	-	deposito_04	D04	temp_d04	high
3	2026-03-18 10:35:00	lector_tarjeta_d07	access	WARNING	U020	deposito_07	D07	lector_d07	deni
4	2026-04-22 11:40:00	sensor_hum_d10	environment	WARNING	-	deposito_10	D10	hum_d10	high
5	2026-05-02 02:14:10	lector_tarjeta_d07	access	CRITICAL	U009	deposito_07	D07	lector_d07	deni
6	2026-05-01 08:00:12	torno_principal	access	INFO	U001	acceso_polvorin	-	torno_01	allo
7	2026-05-01 08:05:40	lector_tarjeta_d01	access	INFO	U001	deposito_01	D01	lector_d01	allo
8	2026-05-01 08:07:15	sensor_magnetico_d01	intrusion	WARNING	-	deposito_01	D01	magnetico_d01	open
9	2026-05-01 08:07:20	volumetrico_d01	intrusion	WARNING	-	acceso_deposito_01	D01	volumetrico_d01	dete
10	2026-05-01 08:09:20	sensor_temp_d01	environment	WARNING	-	deposito_01	D01	temp_d01	high
11	2026-05-01 08:09:30	sensor_hum_d01	environment	WARNING	-	deposito_01	D01	hum_d01	high

Figura C.1: Contenido de la tabla *events* almacenada en SQLite.

id	alert_type	severity	risk_score	status	deposit_id	device_id	message
1	temperature_high_alert	WARNING	4	OPEN	D04	temp_d04	Temperatura elevada en el depósito D...
2	humidity_high_alert	WARNING	4	OPEN	D10	hum_d10	Humedad elevada en el depósito D10: -
3	door_opened	CRITICAL	8	OPEN	D01	magnetico_d01	Apertura detectada en el depósito D01
4	movement_detected	CRITICAL	8	OPEN	D01	volumetrico_d01	Movimiento detectado en acceso_depos...
5	temperature_high_alert	WARNING	4	OPEN	D01	temp_d01	Temperatura elevada en el depósito D...
6	humidity_high_alert	WARNING	4	OPEN	D01	hum_d01	Humedad elevada en el depósito D01: -
7	system_failure	ERROR	7	OPEN	D01	modulo_d01	Fallo del sistema en el dispositivo -
8	power_failure	CRITICAL	10	OPEN	-	red_general	Fallo crítico en el sistema eléctrico
9	temperature_high_alert	WARNING	4	OPEN	D07	temp_d07	Temperatura elevada en el depósito D...

Figura C.2: Contenido de la tabla *alerts* almacenada en SQLite.

C.3. Diseño arquitectónico

La arquitectura del sistema se ha diseñado siguiendo un enfoque modular con el objetivo de facilitar el mantenimiento y la ampliación futura de la aplicación.

La Figura C.3 muestra la arquitectura general del sistema y la interacción existente entre los diferentes módulos implementados.

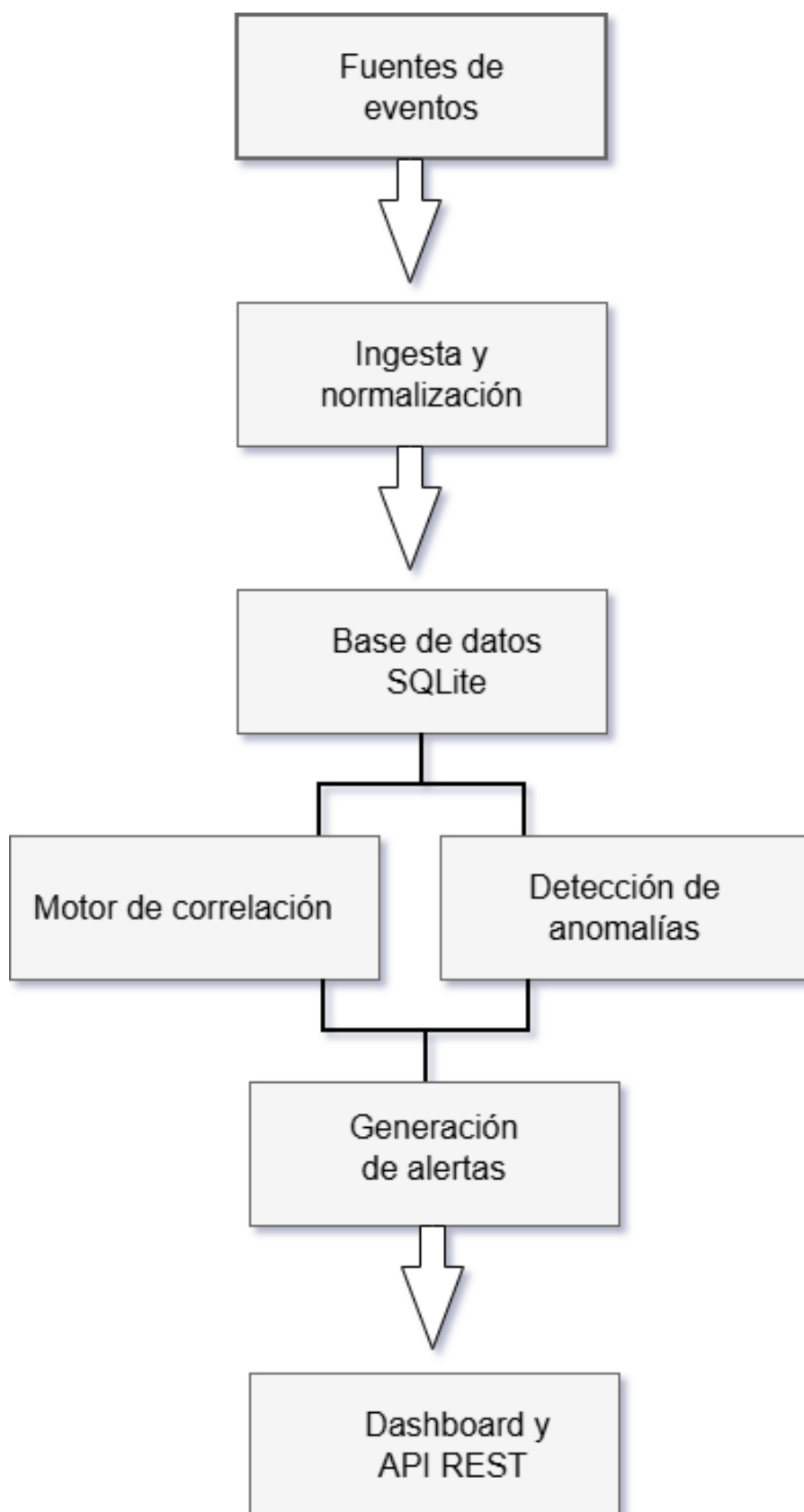


Figura C.3: Arquitectura general del sistema SIEM desarrollado.

La arquitectura implementada sigue un flujo secuencial de procesamiento de la información. Los eventos generados por los diferentes dispositivos simulados son inicialmente procesados por el módulo de ingesta, encargado de leer y normalizar los registros almacenados en los archivos de log.

Una vez normalizados, los eventos son almacenados en la base de datos SQLite, que actúa como repositorio central de información para el resto de componentes del sistema.

A partir de los datos almacenados, el motor de correlación y el módulo de detección de anomalías realizan análisis independientes con el objetivo de identificar situaciones potencialmente peligrosas y generar las alertas correspondientes.

La información obtenida puede ser consultada mediante la API REST desarrollada con FastAPI y visualizada a través del dashboard implementado con Streamlit, proporcionando una visión centralizada del estado del sistema.

Esta separación de responsabilidades permite desacoplar los distintos módulos y simplifica la incorporación de nuevas funcionalidades.

C.4. **Diseño procedimental**

El funcionamiento del sistema sigue una secuencia de procesamiento claramente definida.

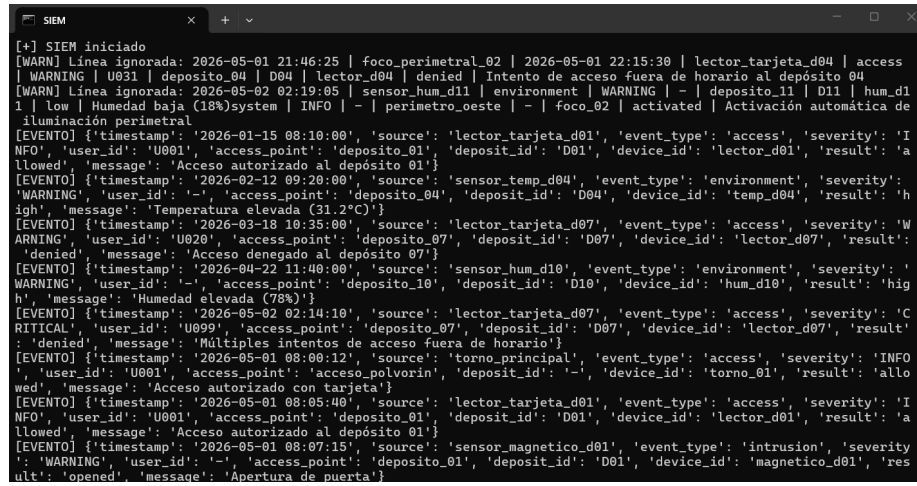
En primer lugar, los dispositivos simulados generan eventos que son almacenados en archivos de logs. El módulo de ingesta procesa dichos eventos y verifica que cumplen el formato establecido.

Posteriormente, los eventos son almacenados en la base de datos y analizados por el motor de correlación y el sistema de detección de anomalías. Cuando se detecta una situación relevante, el sistema genera la correspondiente alerta.

Finalmente, la información almacenada puede ser consultada mediante la API REST y visualizada desde el dashboard de monitorización.

Este flujo de trabajo permite procesar de forma ordenada toda la información generada por el entorno monitorizado y facilita la supervisión continua del sistema.

La Figura C.4 muestra una ejecución real del sistema durante las pruebas realizadas, donde puede observarse el procesamiento de eventos y la generación de alertas por parte de los distintos módulos.



```

[+] SIEM iniciado
[WARN] Línea ignorada: 2026-05-01 21:46:25 | foco_perimetral_02 | 2026-05-01 22:15:30 | lector_tarjeta_d04 | access
| WARNING | U031 | deposito_04 | D04 | lector_d04 | denied | Intento de acceso fuera de horario al depósito 04
[WARN] Línea ignorada: 2026-05-02 02:19:05 | sensor_hum_d11 | environment | WARNING | - | deposito_11 | D11 | hum_d1
1 | low | Humedad baja (18%)system | INFO | - | perimetro_oeste | - | foco_02 | activated | Activación automática de
iluminación perimetral
[EVENTO] {'timestamp': '2026-01-15 08:10:00', 'source': 'lector_tarjeta_d01', 'event_type': 'access', 'severity': 'I
NFO', 'user_id': 'U001', 'access_point': 'deposito_01', 'deposit_id': 'D01', 'device_id': 'lector_d01', 'result': 'a
llowed', 'message': 'Acceso autorizado al depósito 01'}
[EVENTO] {'timestamp': '2026-02-12 09:20:00', 'source': 'sensor_temp_d04', 'event_type': 'environment', 'severity':
'WARNING', 'user_id': '-', 'access_point': 'deposito_04', 'deposit_id': 'D04', 'device_id': 'temp_d04', 'result': 'h
igh', 'message': 'Temperatura elevada (31.2°C)'}
[EVENTO] {'timestamp': '2026-03-18 10:35:00', 'source': 'lector_tarjeta_d07', 'event_type': 'access', 'severity': 'W
ARNING', 'user_id': 'U020', 'access_point': 'deposito_07', 'deposit_id': 'D07', 'device_id': 'lector_d07', 'result':
'denied', 'message': 'Acceso denegado al depósito 07'}
[EVENTO] {'timestamp': '2026-04-22 11:40:00', 'source': 'sensor_hum_d10', 'event_type': 'environment', 'severity': '
WARNING', 'user_id': '-', 'access_point': 'deposito_10', 'deposit_id': 'D10', 'device_id': 'hum_d10', 'result': 'hig
h', 'message': 'Humedad elevada (78%)'}
[EVENTO] {'timestamp': '2026-05-02 02:14:10', 'source': 'lector_tarjeta_d07', 'event_type': 'access', 'severity': 'C
RITICAL', 'user_id': 'U099', 'access_point': 'deposito_07', 'deposit_id': 'D07', 'device_id': 'lector_d07', 'result':
'denied', 'message': 'Múltiples intentos de acceso fuera de horario'}
[EVENTO] {'timestamp': '2026-05-01 08:00:12', 'source': 'torno_principal', 'event_type': 'access', 'severity': 'INFO
', 'user_id': 'U001', 'access_point': 'acceso_polvorin', 'deposit_id': '-', 'device_id': 'torno_01', 'result': 'allo
wed', 'message': 'Acceso autorizado con tarjeta'}
[EVENTO] {'timestamp': '2026-05-01 08:05:40', 'source': 'lector_tarjeta_d01', 'event_type': 'access', 'severity': 'I
NFO', 'user_id': 'U001', 'access_point': 'deposito_01', 'deposit_id': 'D01', 'device_id': 'lector_d01', 'result': 'a
llowed', 'message': 'Acceso autorizado al depósito 01'}
[EVENTO] {'timestamp': '2026-05-01 08:07:15', 'source': 'sensor_magnetico_d01', 'event_type': 'intrusion', 'severity':
'WARNING', 'user_id': '-', 'access_point': 'deposito_01', 'deposit_id': 'D01', 'device_id': 'magnetico_d01', 'res
ult': 'opened', 'message': 'Apertura de puerta'}

```

Figura C.4: Ejecución del sistema SIEM durante el procesamiento de eventos.

Apéndice D

Documentación técnica de programación

D.1. Introducción

El presente anexo tiene como objetivo proporcionar la información técnica necesaria para comprender la estructura interna del sistema desarrollado y facilitar futuras tareas de mantenimiento, ampliación o adaptación del proyecto.

En este documento se describen los principales componentes software que forman parte de la solución implementada, la organización de los directorios del proyecto, los procedimientos de instalación y ejecución, así como las pruebas realizadas para verificar el correcto funcionamiento de los diferentes módulos.

La documentación técnica incluida permite comprender la arquitectura interna del sistema SIEM desarrollado, así como las relaciones existentes entre los diferentes componentes que intervienen en el procesamiento, almacenamiento y visualización de la información.

Asimismo, se proporciona información orientada a futuros desarrolladores que deseen continuar la evolución del proyecto, incorporar nuevas funcionalidades o adaptar el sistema a otros entornos de monitorización. La arquitectura modular utilizada facilita estas tareas y permite realizar modificaciones sobre componentes concretos sin afectar significativamente al funcionamiento global de la aplicación.

D.2. Estructura de directorios

El proyecto se encuentra organizado siguiendo una estructura modular que permite separar claramente las diferentes responsabilidades del sistema.

Esta organización facilita el mantenimiento del código, la incorporación de nuevas funcionalidades y la reutilización de componentes sin afectar al funcionamiento global de la aplicación.

La estructura principal del proyecto es la siguiente:

```
TFG_SIEM/  
|  
+-- api/  
+-- config/  
+-- correlation/  
+-- dashboard/  
+-- db/  
+-- docs/  
+-- ingestor/  
+-- logs/  
+-- machine/  
+-- notifications/  
+-- sensors/  
|  
+-- main.py  
+-- README.md  
+-- requirements.txt  
+-- start.bat
```

La Figura D.1 muestra la estructura general del proyecto visualizada desde el entorno de desarrollo Visual Studio Code.

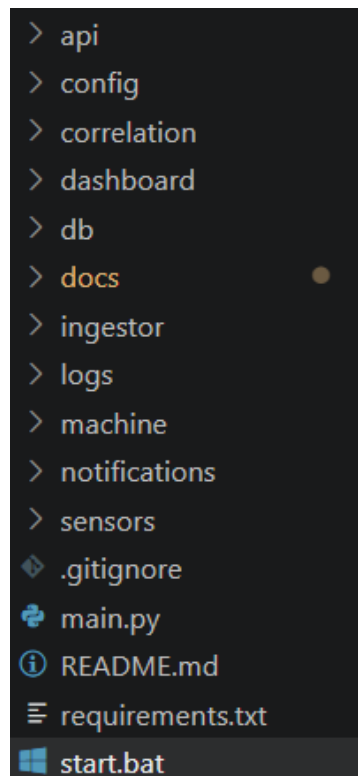


Figura D.1: Estructura general del proyecto visualizada desde Visual Studio Code.

- **api**: Contiene la implementación de la API REST desarrollada con FastAPI.
- **config**: Incluye los parámetros de configuración utilizados por el sistema, tales como umbrales ambientales, rutas de almacenamiento y tiempos de supervisión.
- **correlation**: Implementa el motor de correlación y las reglas encargadas de generar alertas a partir de los eventos procesados.
- **dashboard**: Contiene la aplicación web desarrollada con Streamlit para la visualización de eventos, alertas y anomalías.
- **db**: Gestiona el acceso a la base de datos SQLite y el almacenamiento persistente de eventos y alertas.
- **docs**: Contiene la documentación generada durante el desarrollo del proyecto, incluyendo la memoria y documentación auxiliar.

- **ingestor**: Implementa el módulo encargado de leer y normalizar los logs generados por los dispositivos simulados.
- **logs**: Almacena los archivos de eventos utilizados durante la simulación del entorno monitorizado.
- **machine**: Incluye los mecanismos de detección de anomalías y cálculo de niveles de riesgo.
- **notifications**: Gestiona la simulación de notificaciones asociadas a alertas críticas.
- **sensors**: Implementa la supervisión del estado de los sensores y la detección de dispositivos inactivos o que han dejado de transmitir información.
- **main.py**: Punto de entrada principal de la aplicación. Inicializa los módulos del sistema, ejecuta el proceso de ingesta, lanza las reglas de correlación y coordina el funcionamiento general del SIEM.
- **README.md**: Documento principal de presentación del proyecto donde se describe su funcionamiento, requisitos e instrucciones básicas de instalación y uso.
- **requirements.txt**: Contiene las dependencias necesarias para la instalación y ejecución del proyecto.
- **start.bat**: Script de ejecución utilizado para iniciar automáticamente los principales componentes del sistema en entornos Windows.

La separación de responsabilidades entre módulos permite mantener una arquitectura desacoplada y facilita tanto las tareas de mantenimiento como la incorporación de nuevas funcionalidades. Este diseño modular ha permitido desarrollar y validar cada componente de forma independiente antes de su integración dentro del sistema completo.

D.3. Manual del programador

El desarrollo del proyecto se ha realizado utilizando Python como lenguaje principal de programación. La aplicación se ha diseñado siguiendo una arquitectura modular con el objetivo de facilitar su mantenimiento, escalabilidad y reutilización de componentes.

Cada uno de los módulos implementados dispone de una responsabilidad claramente definida, permitiendo realizar modificaciones o ampliaciones sin afectar significativamente al resto del sistema.

El núcleo de la aplicación se encuentra en el fichero *main.py*, encargado de coordinar la lectura de eventos, el almacenamiento de información, la ejecución del motor de correlación y la detección de anomalías.

Para incorporar nuevas reglas de correlación, el desarrollador debe actuar sobre los módulos contenidos dentro del directorio *correlation*. En este componente se implementan las condiciones encargadas de generar alertas a partir de los eventos procesados por el sistema.

Las nuevas reglas de detección de anomalías pueden añadirse dentro del directorio *machine*, donde se encuentran los mecanismos encargados de analizar comportamientos potencialmente anómalos y asignar los niveles de riesgo correspondientes.

La API REST puede ampliarse incorporando nuevos endpoints dentro del directorio *api*. Gracias a la utilización de FastAPI, la incorporación de nuevos servicios resulta sencilla y permite ampliar las capacidades de integración con aplicaciones externas.

Las funcionalidades relacionadas con la visualización de información se encuentran implementadas en el directorio *dashboard*. Desde este módulo es posible añadir nuevas páginas, gráficos o elementos de monitorización utilizando las capacidades proporcionadas por Streamlit.

Los parámetros generales de funcionamiento se almacenan en el directorio *config*, permitiendo modificar umbrales ambientales, tiempos de supervisión o configuraciones generales sin necesidad de alterar el resto del código fuente.

Esta organización facilita el mantenimiento del proyecto y simplifica la incorporación de futuras mejoras o nuevas funcionalidades.

Módulo de ingesta

El módulo de ingesta es el encargado de procesar los archivos de log generados por los diferentes dispositivos simulados. Su función principal consiste en leer los eventos registrados, normalizar su formato y preparar la información para su posterior almacenamiento y análisis.

La implementación permite realizar lecturas incrementales de los archivos de log, evitando reprocesar eventos ya tratados anteriormente.

La Figura D.2 muestra una ejecución real del módulo de ingesta durante las pruebas realizadas, donde puede observarse la lectura de eventos, la normalización de la información y la detección de líneas con formato incorrecto.

```

PS D:\FG\TFG_SHER\python> python ingestor/ingestor.py
[INFO] Ingestor iniciado
[WARN] Línea ignorada: 2026-05-01 21:46:25 | foco_perimetral_02 | 2026-05-01 22:15:30 | lector_tarjeta_d04 | access | WARNING | U001 | deposito_04 | D04 | lector_d04 | denied | Intento de acceso fuera de horario al depósito 04
[WARN] Línea ignorada: 2026-05-02 02:19:05 | sensor_hum_d11 | environment | WARNING | - | deposito_11 | D11 | hum_d11 | low | Humedad baja (18%)system | INFO | - | perimetro_02 | - | foco_02 | activated | Activación automática de iluminación perimetral
[INFO] Iteración 1: 101 eventos
[timestamp: "2026-01-15 00:10:00", "source": "lector_tarjeta_d01", "event_type": "access", "severity": "INFO", "user_id": "U001", "access_point": "deposito_01", "deposit_id": "D01", "device_id": "lector_d01", "result": "allowed", "message": "Acceso autorizado al depósito 01"}
[timestamp: "2026-02-12 00:20:00", "source": "sensor_tem_d04", "event_type": "environment", "severity": "WARNING", "user_id": "-", "access_point": "deposito_04", "deposit_id": "D04", "device_id": "temp_d04", "result": "high", "message": "temperatura elevada (31.25C)"]

```

Figura D.2: Ejecución del módulo de ingesta durante las pruebas realizadas.

Motor de correlación

El motor de correlación constituye uno de los componentes principales del sistema. Este módulo analiza los eventos normalizados por el ingestor y aplica diferentes reglas de seguridad para identificar situaciones potencialmente peligrosas.

Las reglas implementadas permiten generar alertas ante aperturas de puertas, detección de movimiento, valores ambientales fuera de rango, fallos de comunicación en dispositivos y fallos críticos del sistema eléctrico.

Cada alerta generada incluye información relevante para su posterior análisis, como el tipo de alerta, criticidad, nivel de riesgo, estado, depósito afectado, dispositivo implicado y mensaje descriptivo.

```

class CorrelationEngine:
    def correlate(self, events):
        """Aplicamos reglas de correlación sobre los eventos que recibimos."""
        alerts = []

        # Recorremos todos los eventos recibidos por el motor de correlación
        for event in events:
            event_type = event.get("event_type")
            result = event.get("result")
            severity = event.get("severity")
            deposit_id = event.get("deposit_id")
            device_id = event.get("device_id")
            access_point = event.get("access_point")
            message = event.get("message", "")

            # Regla 1: Apertura de depósito
            if event_type == "intrusion" and result == "opened":
                alerts.append({
                    "alert_type": "door_opened",
                    "severity": "CRITICAL",
                    "risk_score": 8,
                    "status": "OPEN",
                    "deposit_id": deposit_id,
                    "device_id": device_id,
                    "message": f"Apertura detectada en el depósito {deposit_id}"
                })

            # Regla 2: Temperatura fuera de rango
            if event_type == "environment" and "temp" in device_id:
                # Extraemos los valores de temperatura y humedad de los mensajes
                value = self.extract_numeric_value(message)

                if value is not None and value > TEMPERATURE_MAX:
                    alerts.append({
                        "alert_type": "temperature_high_alert",
                        "severity": "WARNING",
                        "risk_score": 4,
                        "status": "OPEN",
                        "deposit_id": deposit_id,
                        "device_id": device_id,
                        "message": f"Temperatura elevada en el depósito {deposit_id}: {value}°C"
                    })

                elif value is not None and value < TEMPERATURE_MIN:
                    alerts.append({
                        "alert_type": "temperature_low_alert",
                        "severity": "WARNING",
                        "risk_score": 4,
                        "status": "OPEN",
                        "deposit_id": deposit_id,
                        "device_id": device_id,
                        "message": f"Temperatura baja en el depósito {deposit_id}: {value}°C"
                    })

```

Figura D.3: Implementación de las reglas principales del motor de correlación.

Gestión de base de datos

La persistencia de la información se realiza mediante SQLite. El componente *DatabaseManager* centraliza todas las operaciones relacionadas con el almacenamiento y recuperación de eventos y alertas.

Esta capa de acceso permite desacoplar la lógica de negocio de la gestión de datos, facilitando futuras migraciones a otros motores de bases de datos.

La base de datos implementada en el proyecto se compone principalmente de las tablas *events* y *alerts*, encargadas de almacenar los eventos procesados por el sistema y las alertas generadas por el motor de correlación.

Las Figuras D.4 y D.5 muestran el contenido de ambas tablas durante las pruebas realizadas.

TABLES											
alerts		timestamp	source	event_type	severity	user_id	access_point	deposit_id	device_id	result	filter
events											
# id timestamp source event_type severity user_id access_point deposit_id device_id result message	1	2026-01-15 08:10:00	lector_tarjeta_d01	access	INFO	U001	deposito_01	D01	lector_d01	allo	high
	2	2026-02-12 09:20:00	sensor_temp_d04	environment	WARNING	-	deposito_04	D04	temp_d04	high	high
	3	2026-03-18 10:35:00	lector_tarjeta_d07	access	WARNING	U020	deposito_07	D07	lector_d07	deni	high
	4	2026-04-22 11:40:00	sensor_hum_d10	environment	WARNING	-	deposito_10	D10	hum_d10	high	high
	5	2026-05-02 02:14:10	lector_tarjeta_d07	access	CRITICAL	U009	deposito_07	D07	lector_d07	deni	high
	6	2026-05-01 08:00:12	torno_principal	access	INFO	U001	acceso_polvorin	-	torno_01	allo	high
	7	2026-05-01 08:05:40	lector_tarjeta_d01	access	INFO	U001	deposito_01	D01	lector_d01	allo	high
	8	2026-05-01 08:07:15	sensor_magnetico_d01	intrusion	WARNING	-	deposito_01	D01	magnetico_d01	open	high
	9	2026-05-01 08:07:20	volumetrico_d01	intrusion	WARNING	-	acceso_deposito_01	D01	volumetrico_d01	dete	high
	10	2026-05-01 08:09:20	sensor_temp_d01	environment	WARNING	-	deposito_01	D01	temp_d01	high	high
	11	2026-05-01 08:09:30	sensor_hum_d01	environment	WARNING	-	deposito_01	D01	hum_d01	high	high

Figura D.4: Contenido de la tabla *events* almacenada en SQLite.

TABLES		alerts							
		id	alert_type	severity	risk_score	status	deposit_id	device_id	message
	# id	1	temperature_high_alert	WARNING	4	OPEN	D04	temp_d04	Temperatura elevada en el depósito D.
	# alert_type	2	humidity_high_alert	WARNING	4	OPEN	D10	hum_d10	Humedad elevada en el depósito D10:
	# severity	3	door_opened	CRITICAL	8	OPEN	D01	magnetico_d01	Apertura detectada en el depósito D01
	# risk_score	4	movement_detected	CRITICAL	8	OPEN	D01	volumetrico_d01	Movimiento detectado en acceso depos.
	# status	5	temperature_high_alert	WARNING	4	OPEN	D01	temp_d01	Temperatura elevada en el depósito D.
	# deposit_id	6	humidity_high_alert	WARNING	4	OPEN	D01	hum_d01	Humedad elevada en el depósito D01:
	# device_id	7	system_failure	ERROR	7	OPEN	D01	modulo_d01	Fallo del sistema en el dispositivo
	# message	8	power_failure	CRITICAL	10	OPEN	-	red_general	Fallo critico en el sistema eléctrico
>	# events	9	temperature_high_alert	WARNING	4	OPEN	D07	temp_d07	Temperatura elevada en el depósito D.

Figura D.5: Contenido de la tabla *alerts* almacenada en SQLite.

Detección de anomalías

El módulo de detección de anomalías analiza la información almacenada con el objetivo de identificar comportamientos que se alejan de los patrones considerados normales.

Para cada anomalía detectada se calcula un nivel de riesgo que posteriormente es utilizado por el dashboard para clasificar y representar los resultados obtenidos.

La detección implementada permite identificar situaciones potencialmente peligrosas relacionadas con accesos no autorizados, movimientos en zonas protegidas, condiciones ambientales fuera de rango y fallos de dispositivos críticos.

La Figura D.6 muestra un ejemplo de las anomalías detectadas por el sistema durante las pruebas realizadas.

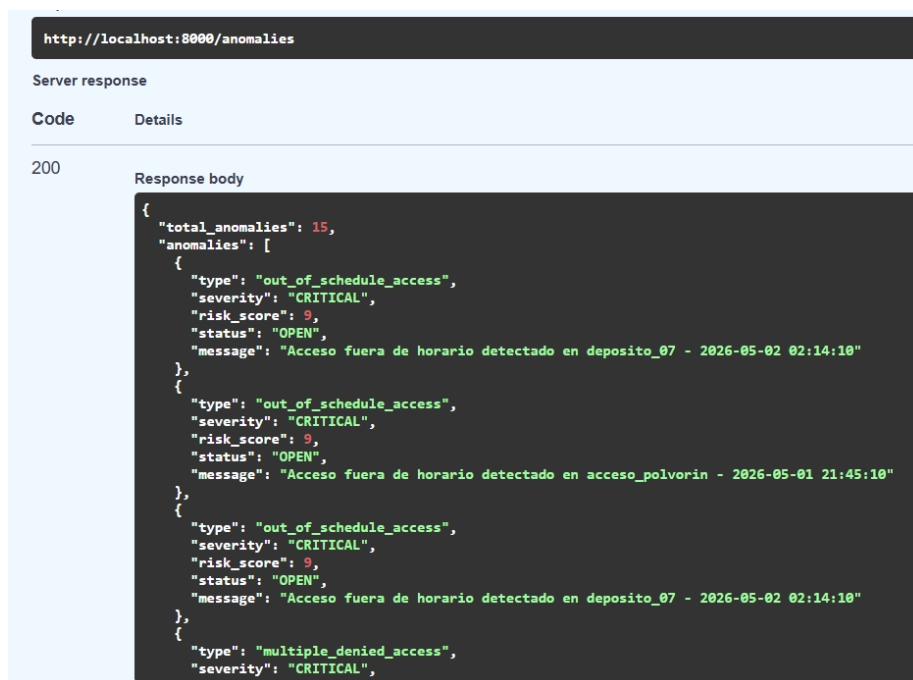


Figura D.6: Resultado de la detección de anomalías clasificadas según su criticidad y nivel de riesgo.

API REST

La API REST desarrollada mediante FastAPI proporciona acceso a la información almacenada por el sistema. Los diferentes endpoints permiten consultar eventos, alertas, anomalías y parámetros de configuración.

La documentación interactiva generada automáticamente mediante Swagger facilita tanto las tareas de desarrollo como la validación de los servicios implementados.

La Figura D.7 muestra la documentación interactiva generada automáticamente por FastAPI, desde la cual es posible consultar y probar los diferentes servicios disponibles.

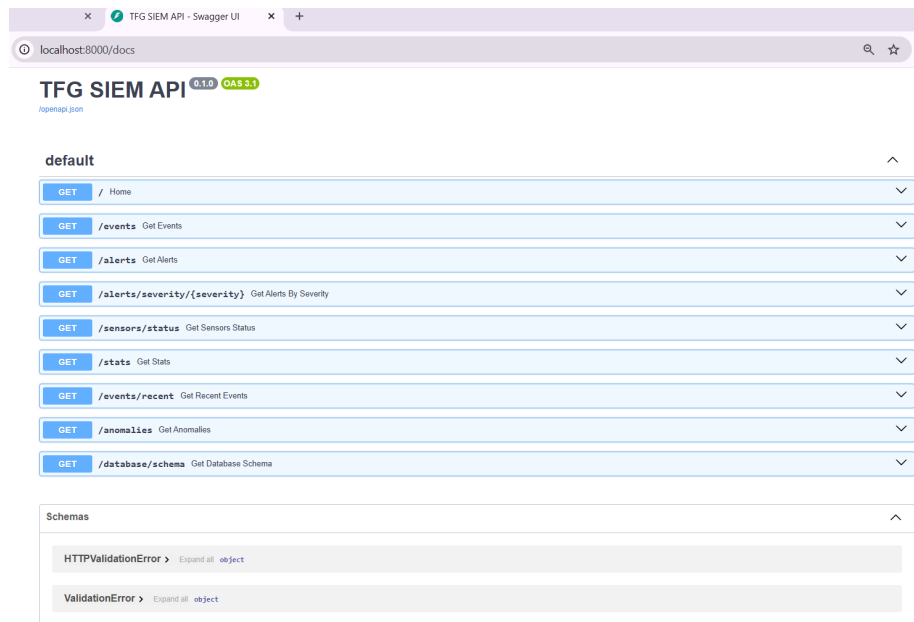


Figura D.7: Documentación interactiva de la API REST generada mediante Swagger.

La Figura D.8 muestra un ejemplo de consulta realizada sobre uno de los endpoints implementados para la recuperación de alertas generadas por el sistema.

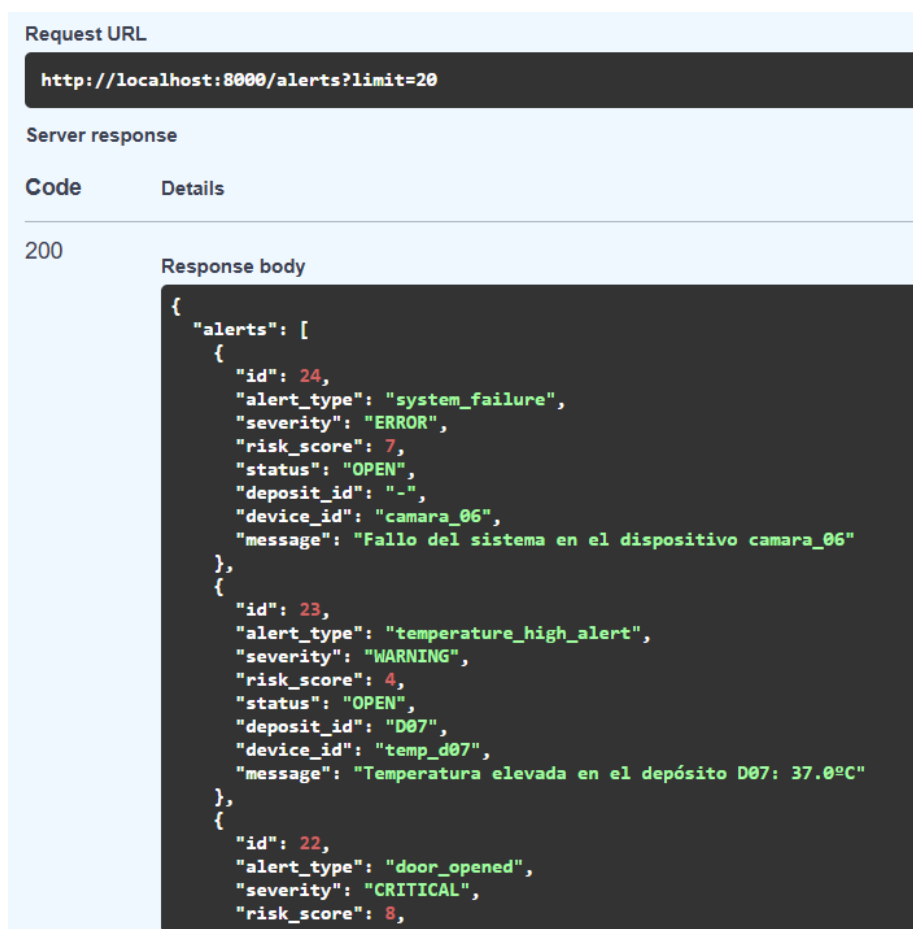


Figura D.8: Consulta realizada mediante el endpoint `/alerts`.

Dashboard de monitorización

La visualización de la información se realiza mediante una aplicación web desarrollada con Streamlit. El dashboard proporciona acceso a eventos, alertas, anomalías, sensores y parámetros de configuración del sistema.

Su diseño permite supervisar de forma centralizada el estado general de la instalación monitorizada y facilita la detección rápida de incidencias.

La Figura D.9 muestra la vista principal del dashboard desarrollado para el proyecto, donde se presentan indicadores generales del estado del sistema y estadísticas de funcionamiento.

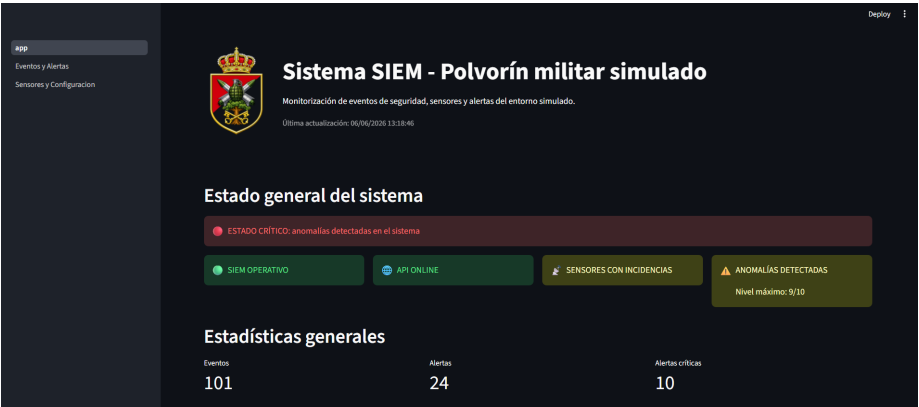


Figura D.9: Vista principal del dashboard de monitorización.

La Figura D.10 muestra una de las secciones destinadas a la consulta de alertas generadas por el sistema durante su funcionamiento.

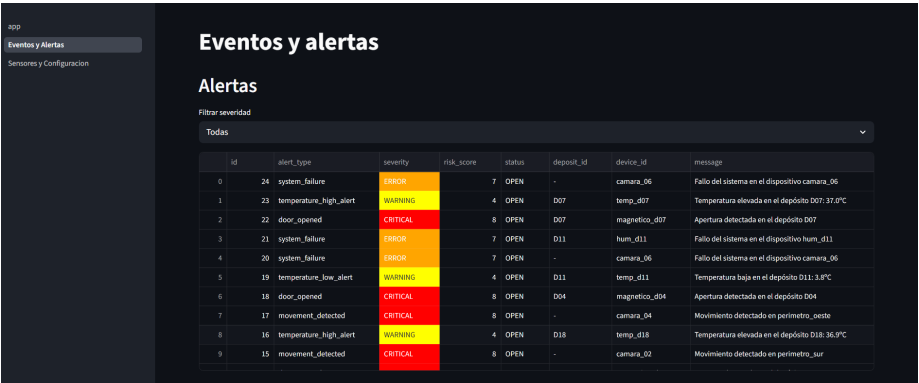


Figura D.10: Visualización de alertas generadas por el sistema.

D.4. Compilación, instalación y ejecución del proyecto

Una vez instaladas las dependencias necesarias, el sistema puede ejecutarse de forma manual o mediante el script de arranque desarrollado para el proyecto.

La ejecución manual de los diferentes componentes se realiza mediante los siguientes comandos:


```
# Ejecución del núcleo principal del SIEM
python main.py
```

```
# Ejecución de la API REST
python -m uvicorn api.server:app --reload
```

```
# Ejecución del dashboard web
python -m streamlit run dashboard/app.py
```

Con el objetivo de simplificar la puesta en marcha del sistema se desarrolló el script *start.bat*, que permite iniciar automáticamente el núcleo principal del SIEM, la API REST y el dashboard web desde una única ejecución.

La Figura D.11 muestra la ejecución del script de arranque desarrollado para el proyecto, encargado de iniciar automáticamente los principales componentes del sistema.

```

C:\WINDOWS\system32\cmd. x +
=====
Iniciando SIEM
=====
Sistema iniciado correctamente.
Presione una tecla para continuar . . .

[ALERTA] {'alert_type': 'system_failure', 'severity': 'ERROR', 'risk_score': 7, 'status': 'OPEN', 'deposit_id': 'D11', 'device_id': 'hum_d11', 'message': 'Fallo del sistema en el dispositivo hum_d11'}
[ALERTA] {'alert_type': 'door_opened', 'severity': 'CRITICAL', 'risk_score': 8, 'status': 'OPEN', 'deposit_id': 'D07', 'device_id': 'magnetico_d07', 'message': 'Apertura detectada en el depósito D07'}
[SMS] Enviado a Capitán: Apertura detectada en el depósito D07
[ALERTA] {'alert_type': 'temperature_high_alert', 'severity': 'WARNING', 'risk_score': 4, 'status': 'OPEN', 'deposit_id': 'D07', 'device_id': 'temp_d07', 'message': 'Temperatura elevada en el depósito D07: 37.0°C'}
[ALERTA] {'alert_type': 'system_failure', 'severity': 'ERROR', 'risk_score': 7, 'status': 'OPEN', 'deposit_id': '-', 'device_id': 'camara_06', 'message': 'Fallo del sistema en el dispositivo camara_06'}

API - python -m uvicorn api. x +
INFO: 127.0.0.1:51211 - "GET /events/recent HTTP/1.1" 200 OK
INFO: 127.0.0.1:51212 - "GET /events?limit=200 HTTP/1.1" 200 OK
INFO: 127.0.0.1:51213 - "GET /anomalies HTTP/1.1" 200 OK
INFO: 127.0.0.1:51214 - "GET /sensors/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:51215 - "GET /stats HTTP/1.1" 200 OK
INFO: 127.0.0.1:51216 - "GET /events/recent HTTP/1.1" 200 OK
INFO: 127.0.0.1:51217 - "GET /events?limit=200 HTTP/1.1" 200 OK
INFO: 127.0.0.1:51218 - "GET /anomalies HTTP/1.1" 200 OK
INFO: 127.0.0.1:51219 - "GET /sensors/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:51220 - "GET /stats HTTP/1.1" 200 OK
INFO: 127.0.0.1:51221 - "GET /events/recent HTTP/1.1" 200 OK
INFO: 127.0.0.1:51222 - "GET /events?limit=200 HTTP/1.1" 200 OK
INFO: 127.0.0.1:51223 - "GET /anomalies HTTP/1.1" 200 OK
INFO: 127.0.0.1:51224 - "GET /sensors/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:51225 - "GET /stats HTTP/1.1" 200 OK
INFO: 127.0.0.1:51226 - "GET /events/recent HTTP/1.1" 200 OK
INFO: 127.0.0.1:51227 - "GET /events?limit=200 HTTP/1.1" 200 OK

Dashboard - python -m streamlit x +
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.
2026-06-07 21:59:43.998 Please replace 'use_container_width' with 'width'.
'use_container_width' will be removed after 2025-12-31.
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.
2026-06-07 21:59:54.686 Please replace 'use_container_width' with 'width'.
'use_container_width' will be removed after 2025-12-31.
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.

```

Figura D.11: Ejecución del script de arranque *start.bat*.

Este mecanismo facilita el despliegue y las pruebas del sistema, evitando la necesidad de ejecutar cada componente de forma independiente y tener que tener los comandos de ejecución a mano, lo que lo simplifica significativamente su ejecución.

D.5. Pruebas del sistema

Durante el desarrollo se realizaron diferentes pruebas funcionales con el objetivo de verificar el correcto funcionamiento de cada uno de los módulos implementados y validar la integración entre los distintos componentes del sistema.

Las principales pruebas realizadas fueron:

- Verificación de la lectura y normalización de eventos desde archivos de log.
- Comprobación del almacenamiento correcto de eventos y alertas en la base de datos SQLite.
- Validación de las reglas de correlación implementadas.
- Comprobación de la detección de anomalías y cálculo del nivel de riesgo.
- Verificación de los servicios expuestos mediante la API REST.
- Validación de la visualización de información en el dashboard desarrollado con Streamlit.
- Pruebas de supervisión del estado de sensores y detección de inactividad.
- Validación del mecanismo de simulación de notificaciones SMS para alertas críticas.

Además de las pruebas individuales realizadas sobre cada módulo, también se llevaron a cabo pruebas de integración para verificar el flujo completo de funcionamiento del sistema, desde la generación de eventos hasta su visualización en el dashboard.

Los resultados obtenidos fueron satisfactorios, verificándose el correcto funcionamiento de todos los componentes principales del sistema y la correcta interacción entre ellos.

A continuación se muestran algunas de las evidencias obtenidas durante las pruebas realizadas sobre los diferentes módulos del sistema.

Pruebas del módulo de ingesta

```
PS D:\TFG\TFG_SIEM> python ingestor/ingestor.py
[INFO] Ingestor iniciado
[WARN] Línea ignorada: 2026-05-01 21:46:25 | foco_perimetral_02 | 2026-05-01 22:15:30 | lector_tarjeta_004 | access | WARNING | 0001 | deposito_04 | 004 | lector_004 | denied | Intento de acceso fuera de hor
ario al deposito 04
[WARN] Línea ignorada: 2026-05-02 02:19:05 | sensor_hum_d11 | environment | WARNING | - | deposito_11 | 011 | hum_d11 | low | Humedad baja (18%)system | INFO | - | perimetro_002 | - | foco_02 | activated |
Activación automática de eliminación perimetral
[INFO] Iteración 1: 101 eventos
{"timestamp": "2026-01-15 08:10:00", "source": "lector_tarjeta_001", "event_type": "access", "severity": "INFO", "user_id": "U001", "access_point": "deposito_01", "deposit_id": "001", "device_id": "lector_d0
1", "result": "allowed", "message": "Acceso autorizado al deposito 01"}
{"timestamp": "2026-02-12 09:20:00", "source": "sensor_temp_004", "event_type": "environment", "severity": "WARNING", "user_id": "-", "access_point": "deposito_04", "deposit_id": "004", "device_id": "temp_d0
4", "result": "high", "message": "Temperatura elevada (31.74C)"}

```

Figura D.12: Ejecución del módulo de ingesta durante las pruebas realizadas.

Pruebas de la API REST

La Figura D.12 muestra la documentación generada automáticamente mediante Swagger, utilizada para verificar el correcto funcionamiento de los endpoints implementados.

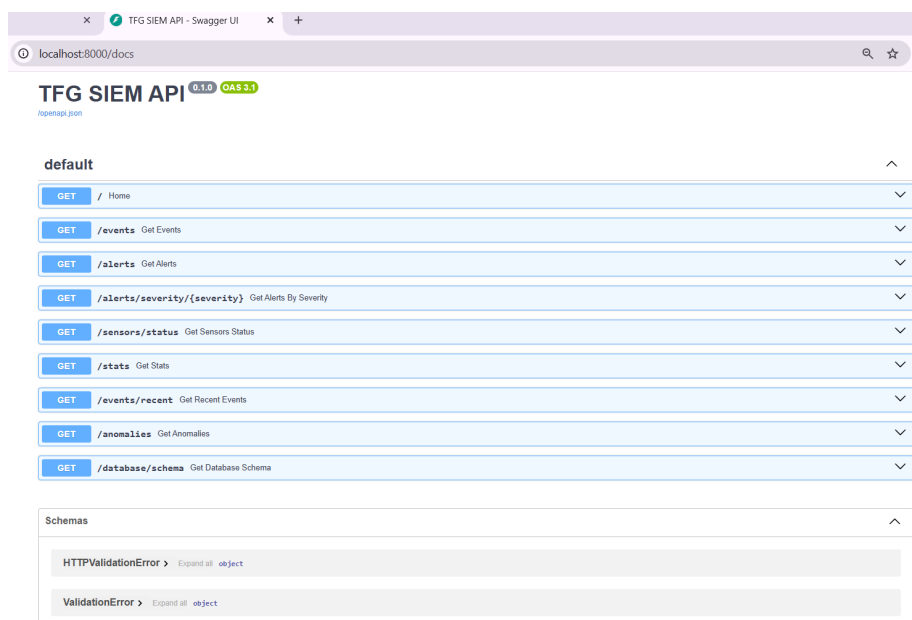


Figura D.13: Documentación Swagger generada automáticamente por FastAPI.

La Figura D.13 muestra una consulta realizada sobre el endpoint de eventos.

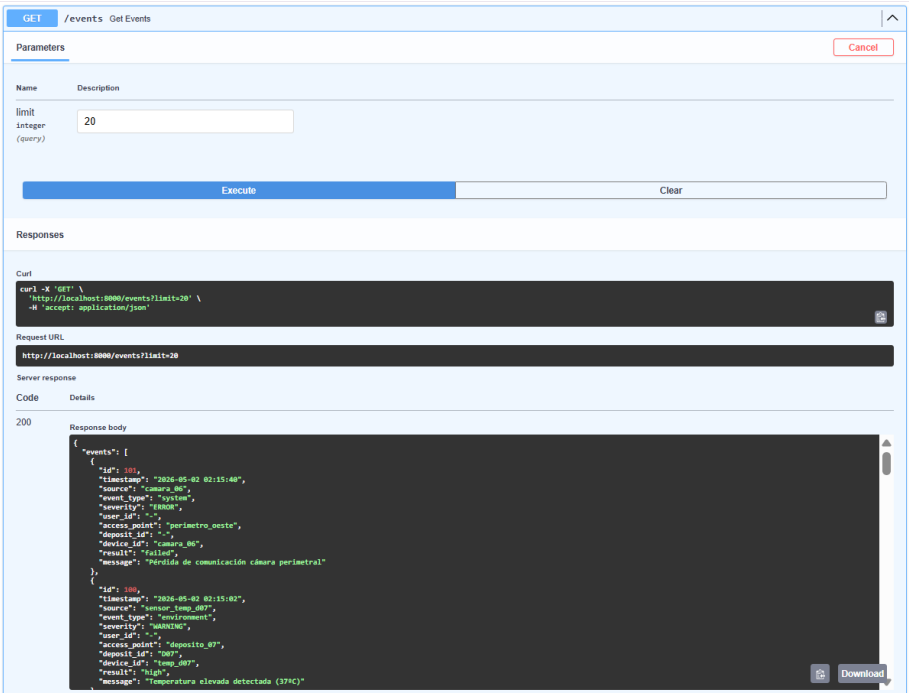


Figura D.14: Consulta de eventos realizada mediante Swagger.

Pruebas de almacenamiento en SQLite

Durante las pruebas se verificó el correcto almacenamiento de eventos y alertas dentro de la base de datos SQLite.

TABLES	Alerts	events	timestamp	source	event_type	severity	user_id	access_point	deposit_id	device_id	result
		1	2026-01-15 08:10:00	lector_tarjeta_d01	access	INFO	U001	deposito_01	D01	lector_d01	allo
		2	2026-02-12 09:20:00	sensor_temp_d04	environment	WARNING	-	deposito_04	D04	temp_d04	high
		3	2026-02-18 10:35:00	lector_tarjeta_d07	access	WARNING	U020	deposito_07	D07	lector_d07	deni
		4	2026-04-22 11:40:00	sensor_hum_d10	environment	WARNING	-	deposito_10	D10	hum_d10	high
		5	2026-05-02 02:14:10	lector_tarjeta_d07	access	CRITICAL	U099	deposito_07	D07	lector_d07	deni
		6	2026-05-01 08:00:12	torro_principal	access	INFO	U001	acceso_pulverin	-	torro_01	allo
		7	2026-05-01 08:05:40	lector_tarjeta_d01	access	INFO	U001	deposito_01	D01	lector_d01	allo
		8	2026-05-01 08:07:15	sensor_magnetico_d01	intrusion	WARNING	-	deposito_01	D01	magnetico_d01	open
		9	2026-05-01 08:07:20	volumetrico_d01	intrusion	WARNING	-	acceso_deposito_01	D01	volumetrico_d01	dete
		10	2026-05-01 08:09:20	sensor_temp_d01	environment	WARNING	-	deposito_01	D01	temp_d01	high
		11	2026-05-01 08:09:30	sensor_hum_d01	environment	WARNING	-	deposito_01	D01	hum_d01	high

Figura D.15: Contenido de la tabla `events` almacenada en SQLite.

id	alert_type	severity	risk_score	status	deposit_id	device_id	message
1	temperature_high_alert	WARNING	4	OPEN	D04	temp_d04	Temperatura elevada en el depósito D...
2	humidity_high_alert	WARNING	4	OPEN	D10	hum_d10	Humedad elevada en el depósito D10: ...
3	door_opened	CRITICAL	8	OPEN	D01	magnetico_d01	Apertura detectada en el depósito D01
4	movement_detected	CRITICAL	8	OPEN	D01	volumetrico_d01	Movimiento detectado en acceso depos...
5	temperature_high_alert	WARNING	4	OPEN	D01	temp_d01	Temperatura elevada en el depósito D...
6	humidity_high_alert	WARNING	4	OPEN	D01	hum_d01	Humedad elevada en el depósito D01: ...
7	system_failure	ERROR	7	OPEN	D01	modulo_d01	Fallo del sistema en el dispositivo ...
8	power_failure	CRITICAL	10	OPEN	-	red_general	Fallo crítico en el sistema eléctrico
9	temperature_high_alert	WARNING	4	OPEN	D07	temp_d07	Temperatura elevada en el depósito D...

Figura D.16: Contenido de la tabla *alerts* almacenada en SQLite.

Pruebas de detección de anomalías

Se verificó el correcto funcionamiento del módulo de detección de anomalías mediante consultas realizadas a través de la API REST.

```

http://localhost:8000/anomalies

Server response

Code    Details
200

Response body
{
  "total_anomalies": 15,
  "anomalies": [
    {
      "type": "out_of_schedule_access",
      "severity": "CRITICAL",
      "risk_score": 9,
      "status": "OPEN",
      "message": "Acceso fuera de horario detectado en deposito_07 - 2026-05-02 02:14:10"
    },
    {
      "type": "out_of_schedule_access",
      "severity": "CRITICAL",
      "risk_score": 9,
      "status": "OPEN",
      "message": "Acceso fuera de horario detectado en acceso_polvorin - 2026-05-01 21:45:10"
    },
    {
      "type": "out_of_schedule_access",
      "severity": "CRITICAL",
      "risk_score": 9,
      "status": "OPEN",
      "message": "Acceso fuera de horario detectado en deposito_07 - 2026-05-02 02:14:10"
    },
    {
      "type": "multiple_denied_access",
      "severity": "CRITICAL",
      "risk_score": 9,
      "status": "OPEN",
      "message": "Acceso denegado en acceso_polvorin - 2026-05-01 21:45:10"
    }
  ]
}

```

Figura D.17: Resultado de la detección de anomalías clasificadas según su nivel de criticidad y nivel de riesgo.

Pruebas del dashboard

También se validó la correcta visualización de la información a través del dashboard desarrollado con Streamlit.

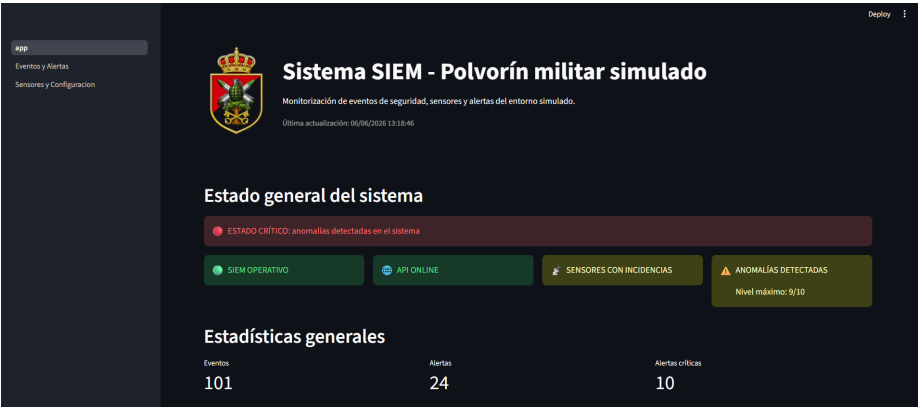


Figura D.18: Vista principal del dashboard.

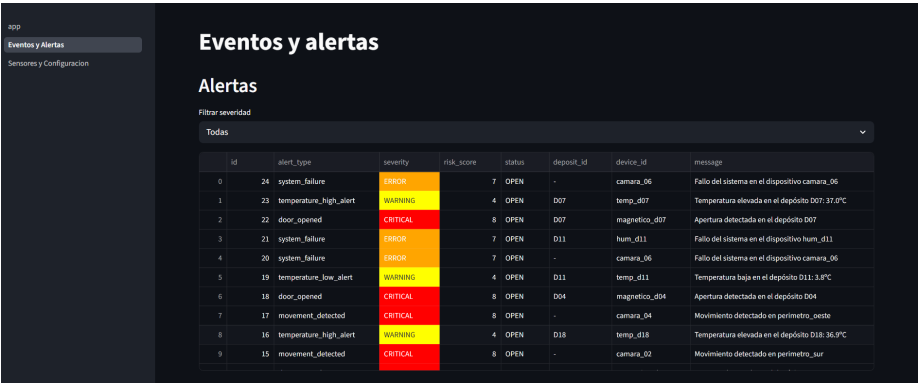


Figura D.19: Visualización de alertas generadas por el sistema.

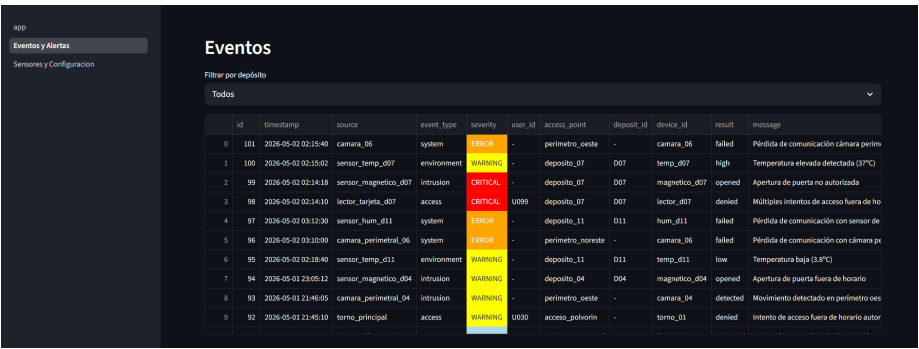


Figura D.20: Visualización de eventos almacenados.

Análisis de calidad del código

Se utilizó la herramienta Codacy para realizar un análisis estático del código fuente y detectar posibles incidencias relacionadas con estilo, documentación, complejidad y buenas prácticas.

Issues breakdown

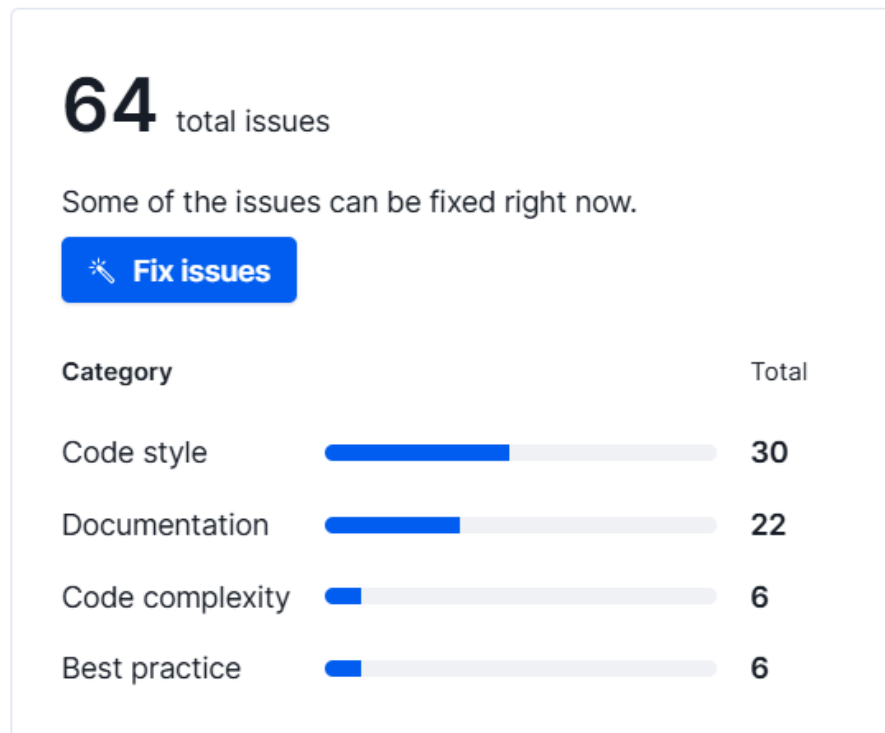


Figura D.21: Resultado del análisis estático realizado mediante Codacy.

Los resultados obtenidos durante las pruebas permitieron verificar el correcto funcionamiento de todos los módulos implementados y confirmar la integración adecuada entre los diferentes componentes que forman parte del sistema SIEM.

Apéndice E

Documentación de usuario

E.1. Introducción

El presente manual describe el procedimiento de utilización del sistema SIEM desarrollado durante este Trabajo Fin de Grado.

La aplicación permite monitorizar eventos de seguridad procedentes de distintos dispositivos simulados, visualizar alertas generadas por el motor de correlación, consultar anomalías detectadas y supervisar el estado de los sensores desplegados dentro del entorno simulado.

El acceso a la información se realiza mediante un dashboard web desarrollado con Streamlit, mientras que la comunicación con la base de datos se lleva a cabo a través de una API REST implementada con FastAPI.

E.2. Requisitos de usuarios

Para utilizar el sistema es necesario disponer de un equipo con sistema operativo Windows, Linux o macOS que tenga instalado Python 3.10 o superior.

Asimismo, es necesario disponer de las dependencias utilizadas por el proyecto, principalmente FastAPI, Streamlit y las bibliotecas auxiliares incluidas en el fichero de requisitos.

Los requisitos hardware son reducidos, ya que el sistema ha sido diseñado como un SIEM ligero. El proyecto puede ejecutarse en equipos de características modestas e incluso en plataformas de bajo consumo como Raspberry Pi.

También se requiere un navegador web moderno para acceder al dashboard de monitorización y a la documentación de la API.

E.3. Instalación

Para instalar el proyecto es necesario descargar el código fuente desde el repositorio y situarse en el directorio principal del proyecto.

Una vez descargado el proyecto, se recomienda crear un entorno virtual de Python e instalar las dependencias necesarias.

Tras la instalación de las dependencias, el sistema puede ponerse en funcionamiento ejecutando el núcleo principal del SIEM, la API REST y el dashboard de monitorización.

La aplicación web estará disponible a través del navegador y permitirá consultar toda la información generada por el sistema.

También es posible iniciar todos los componentes del sistema mediante el script *start.bat*, que automatiza el arranque del núcleo principal, la API REST y el dashboard web.

E.4. Manual del usuario

El acceso al sistema se realiza mediante el dashboard web desarrollado con Streamlit.

La página principal muestra el estado general del sistema, estadísticas de funcionamiento, anomalías detectadas y actividad reciente. A través de esta vista el usuario puede obtener una visión global del estado del sistema y de la actividad registrada.

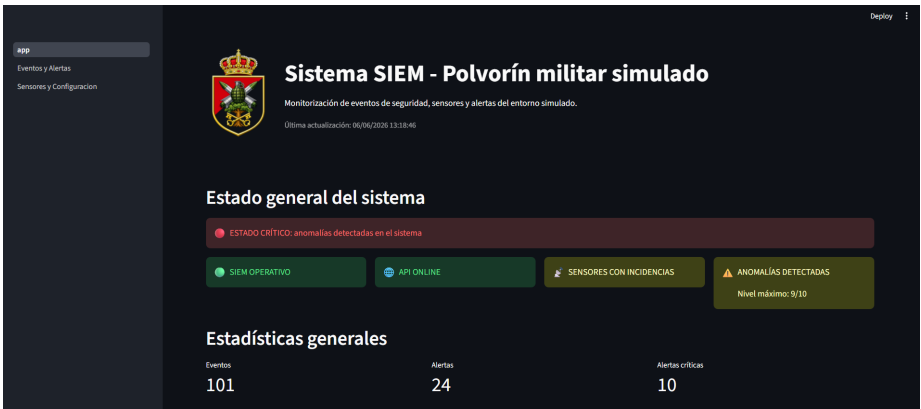


Figura E.1: Vista principal del dashboard del sistema SIEM.

0.90

La página de eventos y alertas permite consultar los registros almacenados por el sistema. El usuario puede utilizar los filtros disponibles para localizar información concreta y analizar la actividad registrada.



Figura E.2: Consulta de alertas generadas por el sistema.

app

Eventos y Alertas

Sensores y Configuración

Eventos

Filterar por depósito

Todos

	id	timestamp	source	event_type	severity	user_id	access_point	deposit_id	device_id	result	message
0	101	2026-05-02 02:15:40	camara_06	system	ERROR	-	perimetro_oeste	-	camara_06	failed	Pérdida de comunicación cámara perim
1	100	2026-05-02 02:15:02	sensor_temp_007	environment	WARNING	-	deposito_07	D07	temp_007	high	Temperatura elevada detectada (37°C)
2	99	2026-05-02 02:14:18	sensor_magnetico_d07	intrusion	CRITICAL	-	deposito_07	D07	magnetico_d07	opened	Apertura de puerta no autorizada
3	98	2026-05-02 02:14:10	lector_tarjeta_d07	access	CRITICAL	U099	deposito_07	D07	lector_d07	denied	Múltiples intentos de acceso fuera de ho
4	97	2026-05-02 03:12:30	sensor_hum_d11	system	ERROR	-	deposito_11	D11	hum_d11	failed	Pérdida de comunicación con sensor de
5	96	2026-05-02 03:10:00	camara_perimetral_06	system	ERROR	-	perimetro_noreste	-	camara_06	failed	Pérdida de comunicación con cámara pe
6	95	2026-05-02 02:18:40	sensor_temp_d11	environment	WARNING	-	deposito_11	D11	temp_d11	low	Temperatura baja (3.8°C)
7	94	2026-05-01 23:05:12	sensor_magnetico_d04	intrusion	WARNING	-	deposito_04	D04	magnetico_d04	opened	Apertura de puerta fuera de horario
8	93	2026-05-01 21:46:05	camara_perimetral_04	intrusion	WARNING	-	perimetro_oeste	-	camara_04	detected	Movimiento detectado en perimetro oes
9	92	2026-05-01 21:45:10	torno_principal	access	WARNING	U030	acceso_polvorin	-	torno_01	denied	Intento de acceso fuera de horario autor

Figura E.3: Consulta de eventos almacenados en la base de datos.

0.90

La página de sensores y configuración muestra el estado de los dispositivos monitorizados y permite consultar determinados parámetros de funcionamiento del sistema.

app

Eventos y Alertas

Sensores y Configuración

Sensores y configuración

Estado de sensores

Total dispositivos: 94

Activos: 40

Inactivos / Sin comunicación: 54

device_id	status	last_event_id
0 temp_d01	active	10.000000
1 hum_d01	active	11.000000
2 magnetico_d01	active	8.000000
3 volumetrico_d01	active	9.000000
4 temp_d02	active	29.000000
5 hum_d02	active	30.000000
6 magnetico_d02	inactive	None
7 volumetrico_d02	inactive	None
8 temp_d03	active	31.000000
9 hum_d03	active	32.000000

Figura E.4: Monitorización del estado de los sensores.

0.90

Las anomalías detectadas aparecen clasificadas según su nivel de riesgo, facilitando la identificación de aquellas situaciones que requieren una atención prioritaria.

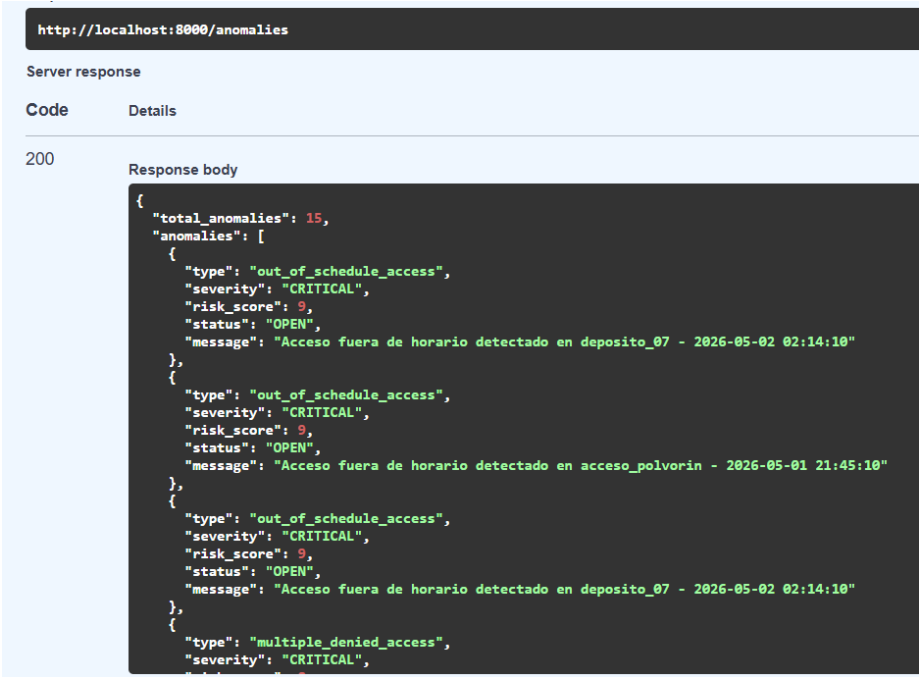


Figura E.5: Visualización de anomalías detectadas por el sistema.

0.90

El dashboard incorpora además diferentes gráficos y estadísticas que permiten visualizar de forma rápida la evolución de los eventos y alertas generados por el sistema.

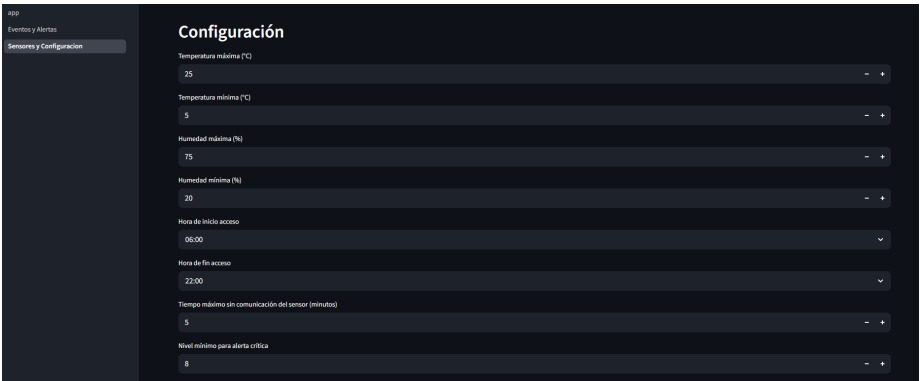


Figura E.6: Panel de configuración del sistema.

0.90



Figura E.7: Configuración de la simulación de notificaciones SMS.

0.90

La combinación de estas funcionalidades permite al usuario supervisar el estado general del sistema, consultar eventos y alertas, analizar anomalías detectadas y configurar distintos parámetros de funcionamiento desde una única interfaz web.

La interfaz desarrollada permite centralizar la supervisión de eventos, alertas, anomalías y sensores en un único punto de acceso, facilitando la monitorización continua del entorno protegido y simplificando las tareas de análisis por parte del usuario.

Anexo de sostenibilización curricular

F.1. Introducción

La sostenibilidad suele asociarse principalmente a aspectos medioambientales, aunque en realidad también incluye dimensiones sociales, económicas y tecnológicas. Durante el desarrollo de este Trabajo Fin de Grado he podido comprobar que muchas de las decisiones adoptadas en un proyecto tecnológico pueden contribuir de forma directa o indirecta a una utilización más eficiente y responsable de los recursos disponibles.

El proyecto desarrollado consiste en un sistema SIEM ligero orientado a la monitorización de eventos de seguridad dentro de un entorno simulado de alta seguridad. Aunque a primera vista pueda parecer un proyecto centrado exclusivamente en aspectos técnicos, durante su desarrollo se han aplicado diversos principios relacionados con la sostenibilidad.

Uno de los aspectos más relevantes ha sido la utilización de tecnologías de código abierto. Herramientas como Python, SQLite, FastAPI, Streamlit, Git o GitHub permiten desarrollar soluciones funcionales sin necesidad de adquirir licencias de software comerciales. Esto favorece la accesibilidad tecnológica y facilita que pequeñas organizaciones o instituciones educativas puedan disponer de herramientas avanzadas sin realizar grandes inversiones económicas.

Otro aspecto importante es la eficiencia en el uso de los recursos hardware. Desde el inicio del proyecto se planteó el desarrollo de un SIEM ligero capaz de ejecutarse en equipos con recursos limitados, incluyendo plataformas

como Raspberry Pi. Este enfoque permite reducir el consumo energético y prolongar la vida útil de los equipos utilizados, contribuyendo a un uso más sostenible de la tecnología.

La sostenibilidad también está relacionada con la seguridad y la continuidad de los servicios. Los sistemas de monitorización permiten detectar incidencias, fallos o situaciones anómalas antes de que se conviertan en problemas de mayor gravedad. En este sentido, el proyecto contribuye a mejorar la supervisión de infraestructuras críticas mediante mecanismos de correlación de eventos y detección de anomalías que facilitan la identificación temprana de posibles riesgos.

Desde el punto de vista social, el desarrollo del proyecto ha permitido adquirir competencias relacionadas con el trabajo autónomo, la resolución de problemas y el aprendizaje continuo. A lo largo del desarrollo ha sido necesario analizar diferentes alternativas tecnológicas, tomar decisiones de diseño y adaptarse a nuevas herramientas y metodologías. Estas capacidades resultan fundamentales para el ejercicio profesional responsable dentro del ámbito de la ingeniería informática.

Asimismo, la utilización de herramientas de inteligencia artificial como apoyo al desarrollo ha permitido mejorar la productividad y agilizar determinadas tareas técnicas y documentales. No obstante, durante el proyecto se ha mantenido una actitud crítica respecto a la información generada por estas herramientas, revisando y validando siempre los resultados obtenidos antes de incorporarlos al sistema.

Por último, considero que este trabajo me ha permitido comprender mejor la importancia de diseñar soluciones tecnológicas eficientes, mantenibles y escalables. La sostenibilidad no debe entenderse únicamente como una cuestión medioambiental, sino también como la capacidad de desarrollar sistemas que aprovechen adecuadamente los recursos disponibles, puedan mantenerse a largo plazo y aporten valor real a las organizaciones y a la sociedad.

Los aspectos abordados durante el desarrollo del proyecto pueden relacionarse especialmente con el Objetivo de Desarrollo Sostenible 9 (Industria, innovación e infraestructura), al promover el desarrollo de soluciones tecnológicas innovadoras basadas en software libre y arquitecturas eficientes. Asimismo, el proyecto guarda relación con el Objetivo de Desarrollo Sostenible 16 (Paz, justicia e instituciones sólidas), ya que contribuye al ámbito de la seguridad y la monitorización de infraestructuras sensibles mediante herramientas que facilitan la detección temprana de incidentes y la mejora de los mecanismos de supervisión.

En conclusión, el desarrollo de este Trabajo Fin de Grado ha contribuido a reforzar mi comprensión de los principios de sostenibilidad aplicados al ámbito tecnológico, mostrando cómo decisiones relacionadas con la arquitectura del sistema, las herramientas utilizadas y la gestión de los recursos pueden tener un impacto positivo desde el punto de vista económico, social y medioambiental.

Bibliografía
